

Indian Institute of Technology Kanpur
Department of Electrical Engineering

Foundation Model for Wireless Communication

Undergraduate Project (UGP) Report

Supervisor

Prof. Rohit Budhiraja

Department of Electrical Engineering
IIT Kanpur

Student

Ronit Kumar

Department of Electrical Engineering
IIT Kanpur

Student

Udbhav Agarwal

Department of Electrical Engineering
IIT Kanpur

May 2026

Contents

1	Introduction	2
1.1	Objectives and Scope	3
2	Background	4
2.1	The Large Wireless Model	4
2.2	Beam Prediction Task	5
2.3	Evaluation Protocol	5
3	Phase 1: Downstream Head Improvement ResCNN to SECNN	6
3.1	Baseline: Residual 1-D CNN (ResCNN)	6
3.2	Squeeze-and-Excitation Mechanism	6
3.3	SE-Residual Block	7
3.4	SECNN Architecture (SEResNet1D)	8
3.5	Training Setup	8
3.6	Results and Discussion	9
3.7	Codebase	10
4	Phase 2: Attention Integration in LWM 1.0	11
4.1	Coordinate Attention (CA)	11
4.2	Axial Attention	12
4.3	Physics-guided Attention	13
4.4	Results and Discussion	14
4.5	Codebase	15
5	Phase 3: Coordinate Attention on LWM 1.1	16
5.1	LWM 1.1: Architectural Evolution	16
5.2	CA Integration into LWM 1.1	17
5.3	Results and Discussion	17
5.4	Codebase	18
6	Phase 4: Mask-Aware Transformer	19
6.1	Architecture: MAT-Pseudo (8×16)	19
6.2	Computational Efficiency	20

6.3	Results: MAT-Pseudo vs LWM	20
6.4	CA Integration: MAT-Pseudo-CA	21
6.5	Codebase	23
7	Architectural Exploration and Insights	24
7.1	MAT-Pseudo-Heavy	24
7.2	MAT-ViT (32×32)	25
7.3	MAT-ViT-Heavy (32×32)	25
7.4	Key Takeaways	26

Abstract

The Large Wireless Model (LWM) [1] is a transformer-based foundation model that is pre-trained in a self-supervised fashion on large-scale wireless channel data using a masked channel modelling (MCM) objective. Although LWM produces rich, context-aware channel embeddings that benefit a variety of downstream tasks, the original system leaves two important dimensions largely unexplored: the design of the downstream prediction head and the computational efficiency of the backbone itself.

This report documents four phases of work carried out during the undergraduate project. In **Phase 1**, we replace the baseline Residual CNN (ResCNN) downstream head with a Squeeze-and-Excitation CNN (SECNN), achieving consistent improvements across all training-data regimes. In **Phase 2**, we integrate three attention mechanisms Coordinate Attention, Axial Attention, and Physics-guided attention directly into the LWM 1.0 encoder and benchmark them on the beam prediction task; Coordinate Attention emerges as the strongest variant. In **Phase 3**, we port Coordinate Attention to the newer LWM 1.1 backbone and compare its performance against LWM 1.0, observing that while LWM 1.1 under-performs LWM 1.0 in the base configuration, the addition of Coordinate Attention narrows the gap noticeably. In **Phase 4**, we explore the Mask-Aware Transformer (MAT) [3] as a lighter-weight backbone, developing four architectural variants; the best variant (MAT-Pseudo) matches or exceeds the base LWM 1.0 at roughly one-half the FLOPs and one-third the parameters.

The present report covers Phase 1 in full detail; subsequent phases will be documented incrementally.

1 Introduction

Current and future wireless communication and sensing systems promise substantial performance gains by relying on massive antenna arrays, high-frequency bands (mmWave and sub-terahertz), and dense network deployments. However, realizing this potential requires overcoming critical challenges in high-dimensional signal processing, massive wireless overhead, and complex network optimization. While deep learning offers powerful data-driven solutions to these problems, traditional models often struggle to generalize across diverse, dynamic wireless environments and heavily depend on large labeled datasets which are typically scarce and expensive to collect.

To address these fundamental limitations, the paradigm of foundation models has recently been adapted to wireless communications. The **Large Wireless Model (LWM)** [1] represents a pioneering step in this direction: it is a transformer-based foundation model pre-trained in a self-supervised manner on a massive corpus of synthetic wireless channels. By employing a *Masked Channel Modelling* (MCM) objective similar to techniques that have revolutionized natural language processing (BERT) and computer vision (ViT), LWM learns to extract rich, universal, and context-aware channel embeddings. These embeddings act as a powerful universal feature extractor that can be transferred to various downstream tasks, significantly improving performance and reducing the reliance on extensive labeled task-specific data.

While LWM successfully demonstrates the viability of foundation models in wireless systems, maximizing its efficiency and downstream capability remains an active area of research. In this project, we focus on enhancing the LWM framework across two main dimensions: feature representation quality and computational efficiency. To rigorously benchmark the impact of our architectural improvements, we utilize *beam prediction*, a critical task for mmWave systems involving the rapid identification of optimal directional beams, as our primary downstream evaluation task.

Despite LWM’s strong baseline performance, several avenues remain open for further improvement:

Downstream head design. The default downstream classifier shipped with LWM is a simple 1-D Residual CNN (ResCNN). This architecture treats all feature channels uniformly and applies aggressive spatial down-sampling. A more expressive downstream head that adaptively weighs important features could extract greater value from LWM’s fixed embeddings, particularly in low-data regimes where extracting maximum information is critical.

Backbone attention mechanisms. The vanilla multi-head self-attention in LWM effectively captures global channel dependencies but lacks explicit inductive biases for coordinate-level or physical structure. Integrating specialized attention modules such

as Coordinate Attention or Physics-guided attention directly into the backbone could yield more informative, structurally-aware embeddings.

Computational efficiency. LWM’s 12-layer transformer encoder is computationally heavy, posing challenges for real-time inference on resource-constrained edge devices. Developing lighter backbone architectures that maintain or exceed prediction quality at a fraction of the computational cost (FLOPs) is essential for practical deployment.

1.1 Objectives and Scope

Motivated by the above considerations, this project pursues two complementary goals: (i) improving prediction quality by upgrading both the downstream head and the backbone attention, and (ii) reducing computational cost by exploring alternative backbone architectures. The work is organised into four phases:

1. **Phase 1: Downstream head improvement.** We design a Squeeze-and-Excitation CNN (SECNN) that incorporates channel attention, preserves spatial resolution, and employs a simpler classifier head. We benchmark it against the original ResCNN on the beam prediction task using both CLS and channel embeddings at six training-data split ratios.
2. **Phase 2: Attention integration in LWM 1.0.** We integrate Coordinate Attention, Axial Attention, and Physics-guided attention into the LWM encoder and evaluate their impact on beam prediction. Coordinate Attention is identified as the best-performing variant.
3. **Phase 3: Experiments with LWM 1.1.** We port Coordinate Attention to the newer LWM 1.1 backbone and compare performance against LWM 1.0, analysing the interplay between backbone version and attention augmentation.
4. **Phase 4: Mask-Aware Transformer backbone.** We develop four MAT-based backbone variants and benchmark them on beam prediction. The MAT-Pseudo variant achieves competitive performance at roughly half the FLOPs and one-third the parameters of LWM 1.0.

2 Background

This section reviews the key building blocks that underpin the experimental work reported in subsequent sections: the LWM architecture, the beam prediction task, and the evaluation methodology.

2.1 The Large Wireless Model

LWM is a 12-layer transformer encoder whose hyper-parameters are summarised in Table 1. The input to the model is a wireless channel matrix $\mathbf{H} \in \mathbb{C}^{M \times N}$ with $M = 32$ base-station antennas and $N = 32$ OFDM sub-carriers.

Table 1: LWM encoder hyper-parameters.

Parameter	Value
Encoder layers	12
Hidden dimension d_{model}	64
Attention heads	12
Head dimension $d_k = d_v$	64/12
Feed-forward dim d_{ff}	256
Patch size L	16
Number of patches P	128
Sequence length (with [CLS])	129
Dropout	0.1

Patch generation. The channel matrix is first separated into its real and imaginary components, $\mathbf{H}_{\text{real}} = \Re(\mathbf{H})$ and $\mathbf{H}_{\text{imag}} = \Im(\mathbf{H})$, each of which is flattened into a vector of length $MN = 1024$. Concatenating these vectors yields a real-valued representation $\mathbf{h} \in \mathbb{R}^{2MN}$ of length 2048. This vector is then divided into $P = 128$ non-overlapping patches of size $L = 16$:

$$\mathbf{p}_i = \mathbf{h}[(i-1)L + 1 : iL], \quad i = 1, \dots, P. \quad (1)$$

Embedding and positional encoding. Each patch $\mathbf{p}_i$ is linearly projected to the model dimension via a learnable weight matrix $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d_{\text{model}} \times L}$, and a learnable positional embedding $\mathbf{e}_i$ is added. A special [CLS] token $\mathbf{p}_0$ is prepended, yielding an input sequence of length 129:

$$\mathbf{z}_i^{(0)} = \mathbf{W}_{\text{proj}} \mathbf{p}_i + \mathbf{e}_i, \quad i = 0, 1, \dots, P. \quad (2)$$

Masked channel modelling. During self-supervised pre-training, 15% of patches are randomly selected and replaced with a learnable [MASK] token (with 10% probability they are instead replaced by a random vector). The encoder processes the corrupted sequence through 12 transformer layers. A lightweight decoder head reconstructs the original patch values at the masked positions, and the model is trained to minimise the mean-squared error between predictions and ground truth.

Embedding extraction. After pre-training, the encoder weights are frozen and **two types of embeddings** are extracted for downstream use. The *CLS embedding* $\text{emb}_{cls} \in \mathbb{R}^{64}$ is the output at the [CLS] position and serves as a global summary of the channel. The *channel embedding* $\text{emb}_{channel} \in \mathbb{R}^{128 \times 64}$, comprising the outputs at all 128 patch positions, preserves fine-grained spatial and spectral structure.

2.2 Beam Prediction Task

The downstream task considered throughout this project is *beam prediction*. A codebook of $N_b = 64$ beams is constructed from a uniform angular grid spanning a 180° field of view. For each user, the steering vector corresponding to each beam angle ϕ_k is applied to the channel, and the beam that maximises the average received power across sub-carriers is selected as the ground-truth label. Including a “no-beam” class, the problem becomes a $(N_b + 1)$ -class classification task.

2.3 Evaluation Protocol

The dataset comprises 519 636 channel samples drawn from 19 urban DeepMIMO scenarios. To study the effect of labelled data availability, we train each downstream model from scratch at six split ratios $r \in \{0.005, 0.01, 0.05, 0.1, 0.2, 0.4\}$, where r denotes the fraction used for training. The remaining $(1 - r)$ fraction is divided equally into validation and test sets. All results are reported as **weighted F1 scores** on the held-out test set, which appropriately accounts for the inherent class imbalance in beam distributions across diverse urban environments.

3 Phase 1: Downstream Head Improvement ResCNN to SECNN

The first phase of the project focuses on improving the downstream prediction head while keeping the LWM encoder frozen. We propose replacing the original Residual CNN (ResCNN) with a Squeeze-and-Excitation CNN (SECNN) that incorporates adaptive channel attention, wider feature maps, and a simplified classifier.

3.1 Baseline: Residual 1-D CNN (ResCNN)

The ResCNN architecture provided with LWM follows a standard residual network design adapted for 1-D sequences. The input embedding (after transposing to channels-first format) passes through a stem convolution with kernel size 7, stride 2, and padding 3, expanding the channel count to 32. Batch normalisation and ReLU activation follow, and a max-pooling layer with kernel size 3 and stride 2 further halves the spatial dimension. Three residual stages then process the features with $\{2, 3, 4\}$ blocks and channel widths $\{32, 64, 128\}$ respectively. Each residual block contains two 3×1 convolutions with batch normalisation; a 1×1 shortcut projection is applied whenever the channel count changes. After global average pooling, two fully-connected layers (128 and N_{classes}) with an intermediate batch normalisation, ReLU, and dropout of 0.5 produce the final logits.

This design has three notable limitations. First, all feature channels are treated uniformly there is no mechanism to emphasise the channels that carry discriminative beam information and suppress those that do not. Second, the stride-2 stem followed by max-pooling aggressively reduces spatial resolution at the very first layer, discarding fine-grained spectral patterns before the network has had a chance to learn which ones matter. Third, the high dropout rate of 0.5, while intended to regularise, can lead to under-fitting when training data is already scarce.

3.2 Squeeze-and-Excitation Mechanism

The Squeeze-and-Excitation (SE) block addresses the first limitation by adaptively recalibrating channel-wise feature responses. Given a feature map $\mathbf{U} \in \mathbb{R}^{C \times L}$ with C channels and spatial length L , the SE block operates in three stages, illustrated in Figure 1.

Squeeze. A global average pooling operation compresses the spatial dimension into a channel descriptor $\mathbf{z} \in \mathbb{R}^C$:

$$z_c = \frac{1}{L} \sum_{l=1}^L u_{c,l}, \quad c = 1, \dots, C. \quad (3)$$

This operation aggregates global spatial information into each channel, providing the excitation stage with a receptive field that spans the entire input length.

Excitation. A lightweight two-layer fully-connected bottleneck learns non-linear inter-channel dependencies and produces a vector of channel attention weights $\mathbf{s} \in \mathbb{R}^C$:

$$\mathbf{s} = \sigma(\mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{z})), \quad (4)$$

where $\mathbf{W}_1 \in \mathbb{R}^{(C/r) \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times (C/r)}$ are learnable weight matrices, $r = 16$ is the reduction ratio that controls the bottleneck width, $\delta(\cdot)$ denotes the ReLU activation, and $\sigma(\cdot)$ denotes the sigmoid function.

Scale. The original feature map is re-weighted channel-wise by the learned attention:

$$\tilde{u}_{c,l} = s_c \cdot u_{c,l}, \quad \forall c, l. \quad (5)$$

Channels deemed important by the excitation network are amplified, while irrelevant or noisy channels are suppressed.

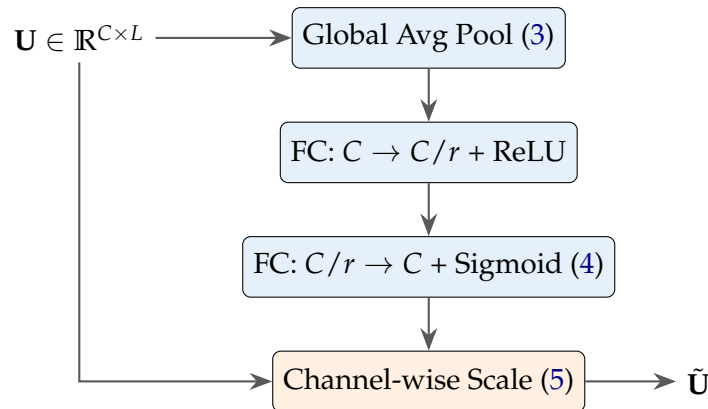


Figure 1: The Squeeze-and-Excitation (SE) block with equation references. The squeeze stage (Eq. 3) aggregates spatial information; the excitation stage (Eq. 4) learns channel attention weights; the scale stage (Eq. 5) re-weights the input.

3.3 SE-Residual Block

The SE mechanism is embedded within a standard residual block to form the *SE-Residual block*. The forward computation is:

$$\mathbf{y} = \text{ReLU}\left(\text{SE}\left(\text{BN}\left(\text{Conv}_{3 \times 1}\left(\text{ReLU}\left(\text{BN}\left(\text{Conv}_{3 \times 1}(\mathbf{x})\right)\right)\right)\right)\right) + \text{shortcut}(\mathbf{x})\right), \quad (6)$$

where each $\text{Conv}_{3 \times 1}$ is a 1-D convolution with kernel size 3, and the shortcut path uses a 1×1 convolution with batch normalisation whenever the stride or channel count changes between input and output.

3.4 SECNN Architecture (SEResNet1D)

The complete SECNN architecture is depicted in Figure 2. It differs from the baseline ResCNN in several important respects.

The stem layer uses a convolution with stride 1 (rather than stride 2) and no subsequent max-pooling, thereby preserving the full spatial resolution of the input through the first residual stage. The initial channel width is set to 64 (doubled from ResCNN’s 32), and the network progressively widens to 128 and then 256 channels across three stages with two residual blocks each.

SE attention is applied selectively: Layer 1 uses plain residual blocks (without SE) to avoid over-regularising low-level features, while Layers 2 and 3 incorporate SE blocks to recalibrate the higher-level, more abstract feature channels. The classifier head is simplified to a single fully-connected layer preceded by a light dropout of 0.1 (compared to ResCNN’s two FC layers and 0.5 dropout). Table 2 summarises the key design differences between the two downstream heads.

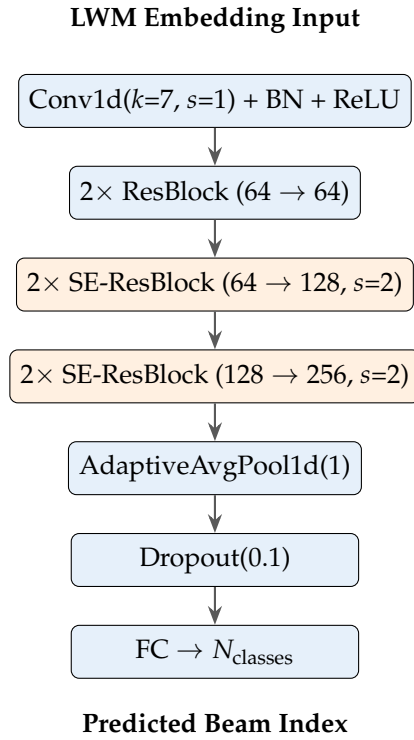


Table 2: Side-by-side comparison of ResCNN and SECNN design choices.

Design Choice	ResCNN	SECNN
Stem stride	2 (+ MaxPool)	1 (no pooling)
Initial channel width	32	64
Blocks per stage	2, 3, 4	2, 2, 2
Channel widths	32, 64, 128	64, 128, 256
SE attention	None	Layers 2 & 3
FC layers after pool	2 (128 → 65)	1 (256 → 65)
Dropout rate	0.5	0.1

Figure 2: SECNN (SEResNet1D) architecture.

3.5 Training Setup

Both models are trained under identical conditions to ensure a fair comparison. The Adam optimiser is used with an initial learning rate of 10^{-3} . A MultiStepLR scheduler reduces the learning rate by a factor of $\gamma = 0.1$ at epochs 15 and 35. The loss function is

standard cross-entropy, and gradient norms are clipped to a maximum of 1.0 to stabilise training. Each configuration is trained for 30 epochs with a batch size of 32. The full dataset of 519 636 samples from 19 DeepMIMO city scenarios is used, with experiments repeated at six split ratios as described in Section 2.3.

3.6 Results and Discussion

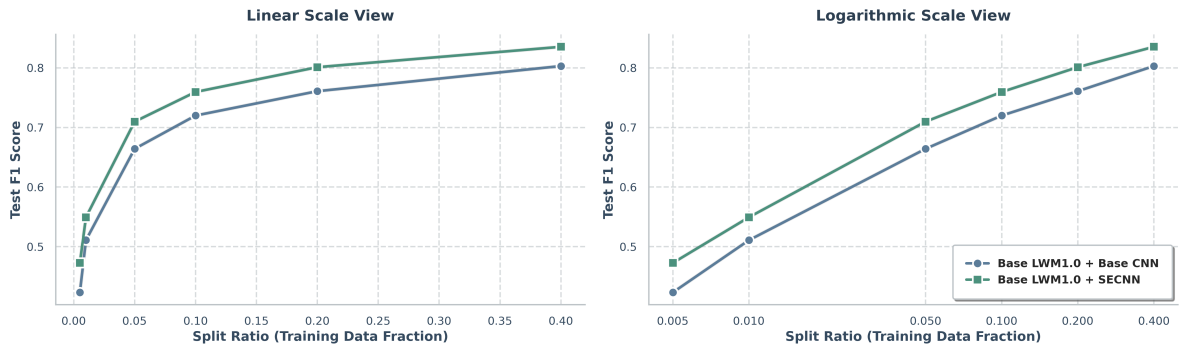
Table 3 reports the test F1 scores using both CLS (emb_{cls}) and channel ($\text{emb}_{\text{channel}}$) embeddings across six split ratios.

Table 3: Test F1 scores on beam prediction. Δ denotes the SECNN improvement in percentage points (pp).

Ratio (r)	CLS Embedding ($\mathbb{R}^{64}$)						Channel Embedding ($\mathbb{R}^{128 \times 64}$)					
	0.5%	1%	5%	10%	20%	40%	0.5%	1%	5%	10%	20%	40%
ResCNN	0.4231	0.5108	0.6642	0.7200	0.7609	0.8031	0.7815	0.8361	0.9082	0.9296	0.9394	0.9524
SECNN	0.4726	0.5493	0.7098	0.7596	0.8012	0.8354	0.7810	0.8496	0.9226	0.9384	0.9508	0.9602
Δ (pp)	+4.95	+3.85	+4.56	+3.96	+4.03	+3.23	-0.05	+1.35	+1.44	+0.88	+1.14	+0.78

With CLS embeddings, SECNN consistently outperforms ResCNN, demonstrating a substantial average gain of ~ 4.1 pp. The most significant improvement (+4.95 pp) occurs at the extreme low-data regime ($r = 0.005$), highlighting the SE mechanism’s ability to efficiently extract discriminative features from a compact global summary. For the spatially-resolved channel embeddings, performance is generally higher, and SECNN maintains a consistent edge (average ~ 0.9 pp), peaking at 0.9602 F1 score for $r = 0.4$. The reduced margin compared to the CLS setting is expected, as the channel embedding already carries rich spatial structure, leaving less room for the downstream head’s inductive bias to make a difference.

Using cls Embeddings for Beam Prediction Task



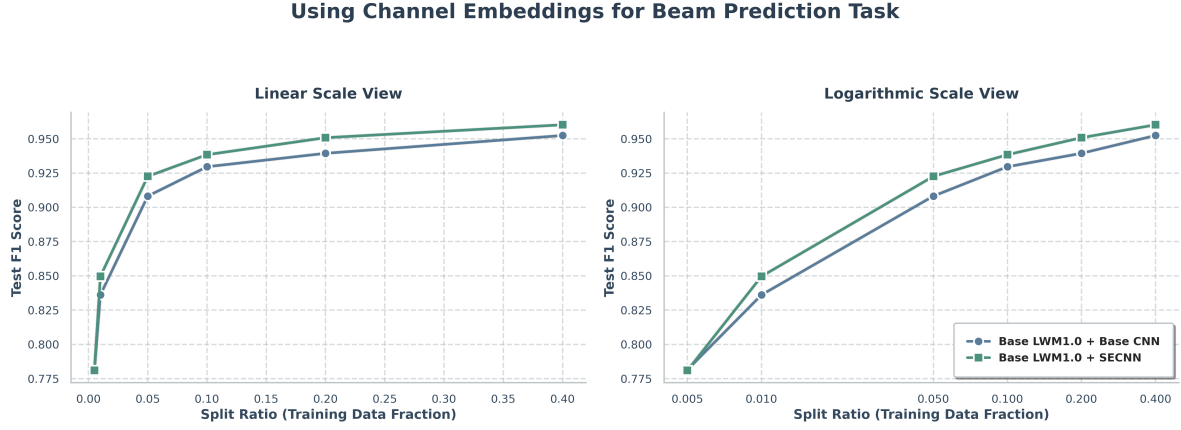


Figure 3: Beam prediction F1 scores. Top: CLS Embeddings. Bottom: Channel Embeddings. SECNN uniformly dominates ResCNN, especially with CLS embeddings.

Inferences. Table 3 highlights that SECNN consistently outperforms ResCNN, with the most significant margin in the low-data regime ($r \leq 0.01$). This underscores the SECNN’s superior feature selection capabilities. By using a stride-1 stem without max-pooling, SECNN maintains a high-fidelity representation of the sparse wireless channel features from the very first layer. This preservation of spatial resolution, combined with the SE module’s ability to “re-calibrate” and focus on discriminative channels, prevents the model from being overwhelmed by noise in the initial LWM embeddings. Furthermore, the reduction in dropout from 0.5 to 0.1 indicates that SECNN’s architecture provides a more effective inductive bias, reducing the need for aggressive stochastic regularization which often leads to underfitting in small-scale wireless datasets.

3.7 Codebase

The implementation for Phase 1 is contained within the `lwm/` directory. These scripts are designed for downstream beam prediction training and inference, utilizing the frozen LWM encoder as a feature extractor. The core logic is distributed as follows:

- `BP_LWM_CNNoSECNN.py`: The primary execution script for downstream training. It defines the `res1dcnn` and `SEResNet1D` (SECNN) architectures and manages training/evaluation using embeddings extracted from the frozen LWM.
- `lwm_model.py`: Defines the transformer-based LWM 1.0 encoder.
- `inference.py`: Handles the forward pass of the frozen LWM backbone to extract channel embeddings, caching results in `.downstream_cache/`.
- `input_preprocess.py`: Manages DeepMIMO data parsing and beam label generation.
- `plotResults.py`: Generates performance plots from `results.txt`.

4 Phase 2: Attention Integration in LWM 1.0

The second phase of the project explores improving the feature extraction capability of the LWM backbone by integrating specialized attention mechanisms. While the vanilla Multi-Head Self-Attention (MHSA) in LWM 1.0 is effective at capturing global dependencies, it treats the input as a flattened sequence, potentially ignoring the inherent 2-D structural relationships (spatial and spectral) present in wireless channels. We integrate three distinct attention paradigms: *Coordinate Attention*, *Axial Attention*, and *Physics-guided Attention*.

4.1 Coordinate Attention (CA)

Coordinate Attention is designed to capture long-range dependencies with precise positional information. Unlike channel attention (like SE) which globalizes spatial information, CA encodes both inter-channel and position-specific information by factorizing 2-D pooling into two 1-D encoding processes.

Mathematical Formulation. Given an input feature map $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ (where H and W correspond to the antenna and frequency dimensions respectively), the process involves:

1. **Coordinate Squeeze:** Global average pooling is performed along the horizontal and vertical axes separately:

$$z_c^h(h) = \frac{1}{W} \sum_{0 \leq i < W} x_c(h, i), \quad z_c^w(w) = \frac{1}{H} \sum_{0 \leq j < H} x_c(j, w) \quad (7)$$

2. **Coordinate Excitation:** The resulting descriptors are concatenated and processed by a shared 1×1 convolution $\mathbf{F}_1$ and non-linear activation δ :

$$\mathbf{f} = \delta(\mathbf{F}_1([\mathbf{z}^h, \mathbf{z}^w])) \quad (8)$$

where $[\cdot, \cdot]$ denotes concatenation along the spatial dimension. $\mathbf{f} \in \mathbb{R}^{C/r \times (H+W)}$ is then split back into $\mathbf{f}^h \in \mathbb{R}^{C/r \times H}$ and $\mathbf{f}^w \in \mathbb{R}^{C/r \times W}$.

3. **Recalibration:** Two 1×1 convolutions $\mathbf{F}_h$ and $\mathbf{F}_w$ transform $\mathbf{f}^h$ and $\mathbf{f}^w$ back to the original channel dimension, followed by a sigmoid σ to produce attention weights:

$$g_c^h = \sigma(\mathbf{F}_h(\mathbf{f}^h)), \quad g_c^w = \sigma(\mathbf{F}_w(\mathbf{f}^w)) \quad (9)$$

The final output is $\tilde{x}_c(i, j) = x_c(i, j) \times g_c^h(i) \times g_c^w(j)$.

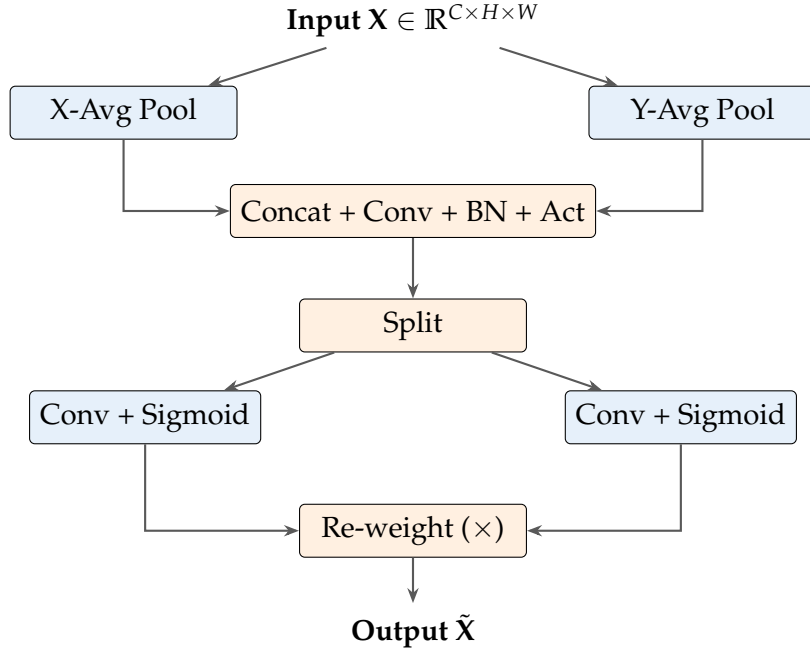


Figure 4: Coordinate Attention mechanism. It factorizes 2D pooling into two 1D encoding processes to capture direction-aware and position-sensitive information.

4.2 Axial Attention

Axial Attention reduces the computational burden of global attention by factorizing it into sequential 1-D attention operations along the different axes of the input. In LWM, this means performing attention separately across the antenna dimension ($H = 32$) and the frequency dimension ($W = 4$).

Mathematical Formulation. For an input grid $X \in \mathbb{R}^{H \times W \times D}$, axial attention computes:

1. **Antenna-axis Attention:** Fixes the frequency index j and performs MHSA over the antenna index i :

$$Y_{i,j}^{ant} = \text{MHSA}_i(X_{i,j}) + X_{i,j} \quad (10)$$

2. **Frequency-axis Attention:** Fixes the antenna index i and performs MHSA over the frequency index j :

$$Y_{i,j}^{freq} = \text{MHSA}_j(Y_{i,j}^{ant}) + Y_{i,j}^{ant} \quad (11)$$

This reduces complexity from $O((H \cdot W)^2)$ to $O(H \cdot W(H + W))$, which is particularly beneficial for high-resolution channel grids. Furthermore, we incorporate **Rotary Positional Embeddings (RoPE)** within each axial operation to inject relative positional information into the attention scores, allowing the model to distinguish between different spatial and spectral lags.

4.3 Physics-guided Attention

Physics-guided attention leverages the underlying physical properties of wireless channels—specifically spatial correlation and frequency coherence—to bias the self-attention mechanism towards physically meaningful relationships.

Physical Prior Bias. We construct a bias matrix $\blacksquare \in \mathbb{R}^{S \times S}$ (where $S = HW + 1$) that encodes the expected correlation between tokens based on their spatial and spectral distances:

$$\Phi_{i,j} = \exp\left(-\frac{|ant_i - ant_j|^2}{2\sigma^2}\right) \cdot \exp\left(-\frac{|freq_i - freq_j|}{\tau_c}\right) \quad (12)$$

where ant_i and $freq_i$ are the antenna and frequency indices of token i respectively. The first term represents the spatial decay over the antenna array, while the second term captures frequency selectivity.

Modified Attention Score. The physics prior is integrated as an additive bias to the attention logits before the softmax operation:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \lambda\blacksquare\right) \mathbf{V} \quad (13)$$

where λ is a learnable scaling factor. This bias encourages the model to focus on clusters of patches that are physically adjacent, reflecting the typical structure of multipath channel components.

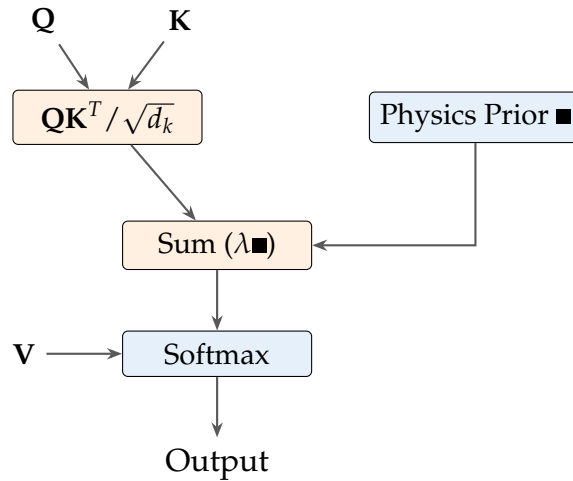


Figure 5: Physics-guided Attention. A physics-derived bias matrix $\blacksquare$ is added to the attention scores to emphasize spatial and spectral locality.

4.4 Results and Discussion

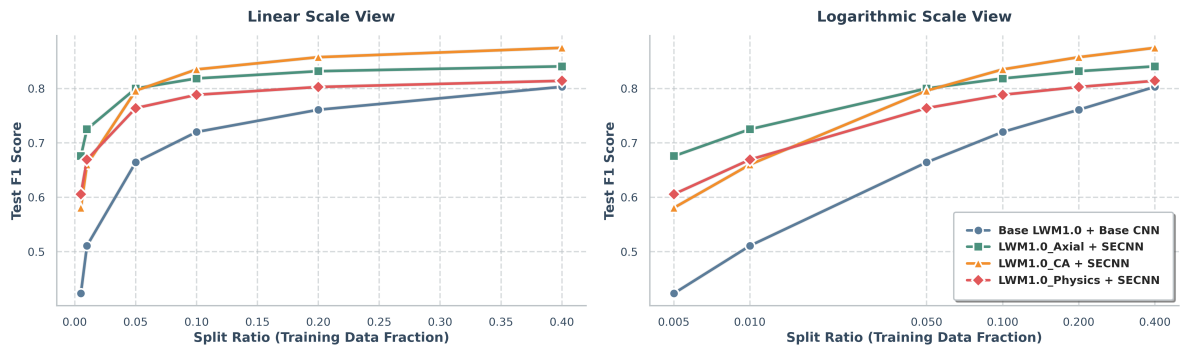
Inferences. Several key insights emerge from the attention variants comparison. First, the superiority of **Coordinate Attention (CA)** on channel embeddings is attributed to its ability to encode precise positional information through direction-aware pooling. Unlike global SE attention which flattens spatial structure, CA factorizes 2D pooling into separate antenna and frequency descriptors, allowing the shared 1×1 convolution to capture inter-dimensional dependencies. This is critical for the 128×64 channel grid where spatial-spectral locality is a dominant feature.

Second, **Axial Attention**'s peak performance at the 0.5% CLS split (0.6756 vs 0.5805 for CA) suggests that its factored MHSA structure acts as a powerful regularizer. By decomposing the global attention into separate antenna and frequency passes, Axial attention significantly reduces the search space for the model, providing a strong structural bias that prevents overfitting when training labels are extremely sparse. Finally, the performance of **Physics-guided Attention** confirms that while physical priors (spatial/spectral decay) provide a solid performance floor, they are ultimately surpassed by data-driven mechanisms like CA that can learn to account for non-idealized channel propagation, such as diffraction and scattering, which are not perfectly captured by the exponential decay prior.

Table 4: Test F1 scores on beam prediction with integrated attention mechanisms.

Ratio (r)	CLS Embedding ($\mathbb{R}^{64}$)						Channel Embedding ($\mathbb{R}^{128 \times 64}$)					
	0.5%	1%	5%	10%	20%	40%	0.5%	1%	5%	10%	20%	40%
LWM_Axial	0.6756	0.7251	0.8000	0.8185	0.8318	0.8407	0.6978	0.7572	0.8923	0.9139	0.9295	0.9402
LWM_CA	0.5805	0.6601	0.7959	0.8351	0.8576	0.8748	0.8196	0.8694	0.9287	0.9410	0.9525	0.9592
LWM_Phys	0.6056	0.6692	0.7639	0.7884	0.8028	0.8141	0.7275	0.8025	0.8979	0.9239	0.9363	0.9451

Using cls Embeddings for Beam Prediction Task



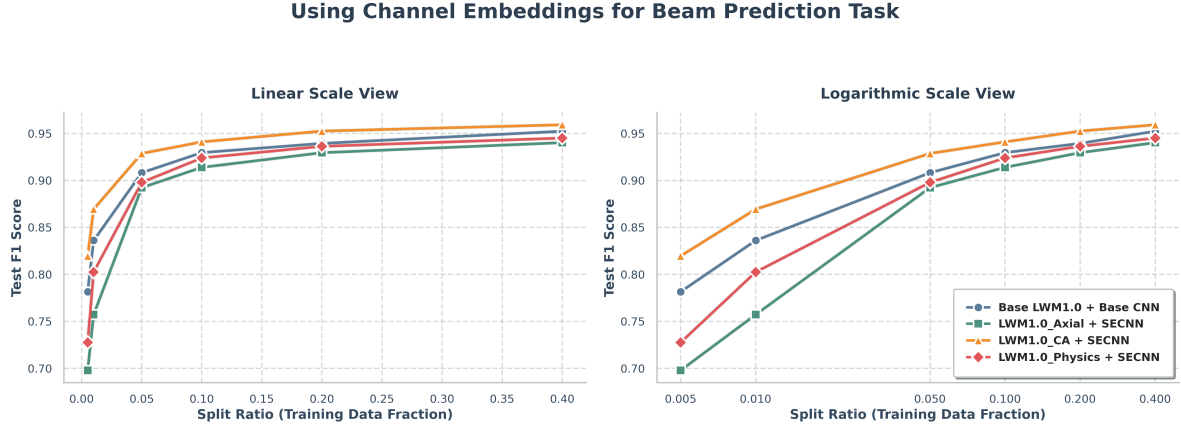


Figure 6: Beam prediction performance comparison for Phase 2 attention variants. Top: CLS Embeddings. Bottom: Channel Embeddings.

4.5 Codebase

Phase 2 development is modularized into three separate folders. Each contains specialized scripts for downstream beam prediction training and inference, leveraging a frozen, attention-augmented LWM backbone:

- `lwm_ca/`: Contains `coordatt.py` (Coordinate Attention block). Downstream training is performed by `BP_CA_LWM_SECNN.py`, which recalibrates the spatial-spectral grid before inferencing from the frozen LWM backbone.
- `lwm_axial/`: Houses `axial_linear_att.py`. The `AxialSoftmaxAttention` module is integrated into the LWM backbone in `lwm_axial_model.py`. `BP_AXL_LWM_SECNN.py` manages the downstream beam prediction tasks using the frozen encoder.
- `lwm_physics/`: Features `physics_priors.py` and `lwm_physics_model.py`. The `BP_PHY_LWM_SECNN.py` script executes the benchmarking pipeline, inferencing from the physics-biased backbone.

Each directory is self-contained with its own `inference.py` and `results.txt`, with all BP_ scripts focusing on training the downstream head using features from the frozen encoder.

5 Phase 3: Coordinate Attention on LWM 1.1

Having established Coordinate Attention (CA) as the best-performing attention variant on LWM 1.0 (Section 4), Phase 3 ports this mechanism to the next-generation backbone, **LWM 1.1** [2], and benchmarks the combined system against both the vanilla LWM 1.0 and the vanilla LWM 1.1 baselines.

5.1 LWM 1.1: Architectural Evolution

LWM 1.1 addresses core limitations in scalability, model capacity, and pre-training diversity relative to version 1.0. Table 5 summarizes the key changes.

Table 5: Comparison of LWM 1.0 and LWM 1.1 specifications.

Feature	LWM 1.0	LWM 1.1
Channel size support	Fixed (32, 32)	Multiple (N, SC) pairs
Max sequence length	128	512
Embedding dim (d_{model})	64	128
Parameters	600 K	2.5 M
Attention heads	12	8
MCM masking ratio	15%	40%
Patch segmentation	1-D (antenna <i>or</i> freq.)	2-D (4×4 ant. $\times$ freq.)
Pre-training samples	820 K	~ 1.3 M
Pre-training scenarios	15	140
Optimizer / schedule	Adam	AdamW + cosine decay

2-D Patch Segmentation. LWM 1.0 flattened the channel along a single axis before tokenization. LWM 1.1 introduces **2-D patches** of size 4×4 (antennas $\times$ subcarriers), producing $n_{\text{patches}} = 64$ tokens from a (32, 32) grid. Each patch vector has length $4 \times 4 \times 2 = 32$ (real + imaginary), yielding a richer spatial-spectral representation that aligns naturally with OFDM channel structure.

Masked Channel Modelling (MCM). Both versions share the MCM self-supervised objective, but LWM 1.1 raises the masking ratio from 15% to **40%**. An 80-10-10 corruption scheme is applied (80% [MASK], 10% random, 10% unchanged). The loss is a variance-normalized MSE:

$$\mathcal{L}_{\text{MCM}} = \frac{\text{MSE}(\mathbf{T}_{\text{mask}}, \hat{\mathbf{T}}_{\text{mask}})}{\text{Var}(\mathbf{T}_{\text{mask}})} \quad (14)$$

where $\mathbf{T}_{\text{mask}}$ and $\hat{\mathbf{T}}_{\text{mask}}$ are the ground-truth and predicted tokens at the masked positions. The increased masking forces the model to rely on deeper spatial-spectral

correlations rather than local context, producing more robust embeddings.

5.2 CA Integration into LWM 1.1

The Coordinate Attention block (described in Section 4.1, Figure 4) is inserted *before* the 2-D patching stage. The raw complex channel $\mathbf{H} \in \mathbb{R}^{2 \times 32 \times 32}$ (real and imaginary stacked) is first recalibrated by the CA module, which produces direction-aware attention maps along the antenna and subcarrier axes. The recalibrated channel is then tokenized into 4×4 patches, masked, and processed by the 12-layer LWM 1.1 transformer. This end-to-end pipeline (`torch_pipeline_v11.py`) is pre-trained jointly via MCM, allowing the CA weights to co-adapt with the backbone.

5.3 Results and Discussion

Table 6 reports the cross-generational comparison across all four configurations. The LWM 1.0 variants use the Base CNN and CA+SECNN heads from Phases 1 and 2 respectively; the LWM 1.1 variants mirror this pairing.

Table 6: Generational comparison of beam prediction F1 scores.

Ratio (r)	CLS Embedding						Channel Embedding					
	0.5%	1%	5%	10%	20%	40%	0.5%	1%	5%	10%	20%	40%
Base LWM 1.0 + Base CNN	0.4231	0.5108	0.6642	0.7200	0.7609	0.8031	0.7815	0.8361	0.9082	0.9296	0.9394	0.9524
LWM 1.0_CA + SECNN	0.5805	0.6601	0.7959	0.8351	0.8576	0.8748	0.8196	0.8694	0.9287	0.9410	0.9525	0.9592
Base LWM 1.1 + Base CNN	0.4003	0.4046	0.5679	0.5758	0.6878	0.7314	0.7119	0.7905	0.8866	0.9105	0.9302	0.9454
LWM 1.1_CA + SECNN	0.4921	0.5686	0.7218	0.7632	0.7823	0.8021	0.7528	0.8164	0.9101	0.9255	0.9400	0.9480

Inferences.

1. **LWM 1.0_CA + SECNN remains the overall leader.** The Phase 2 best configuration dominates across both embedding types and all split ratios (e.g., 0.8748 CLS F1 at $r=0.4$). This confirms that the CA recalibration, when applied to the mature LWM 1.0 backbone with SECNN, produces the strongest downstream features in our evaluation.
2. **LWM 1.1 underperforms LWM 1.0 at low data.** The vanilla LWM 1.1 backbone (2.5 M parameters, 40% masking) trails the compact LWM 1.0 (600 K parameters, 15% masking) across all CLS split ratios and low-data channel splits. The higher-dimensional embedding space ($d=128$ vs $d=64$) requires more downstream supervision to resolve, creating a “cold-start” penalty.
3. **CA recovers LWM 1.1 to competitive levels.** Applying Coordinate Attention to LWM 1.1 yields large gains over the vanilla 1.1 baseline — +9.2 pp at $r=0.005$ and +16.4 pp at $r=0.01$ on CLS embeddings. While it does not surpass LWM 1.0_CA, it closes the gap substantially, demonstrating that CA’s directional priors are essential for unlocking the potential of the larger backbone in low-data regimes.

4. **Channel embeddings converge across generations.** For channel embeddings at $r \geq 0.05$, all four models converge to within ~ 2 pp (0.91–0.96 F1), indicating that the spatial richness of high-dimensional channel embeddings saturates the downstream head regardless of the backbone or attention variant used.

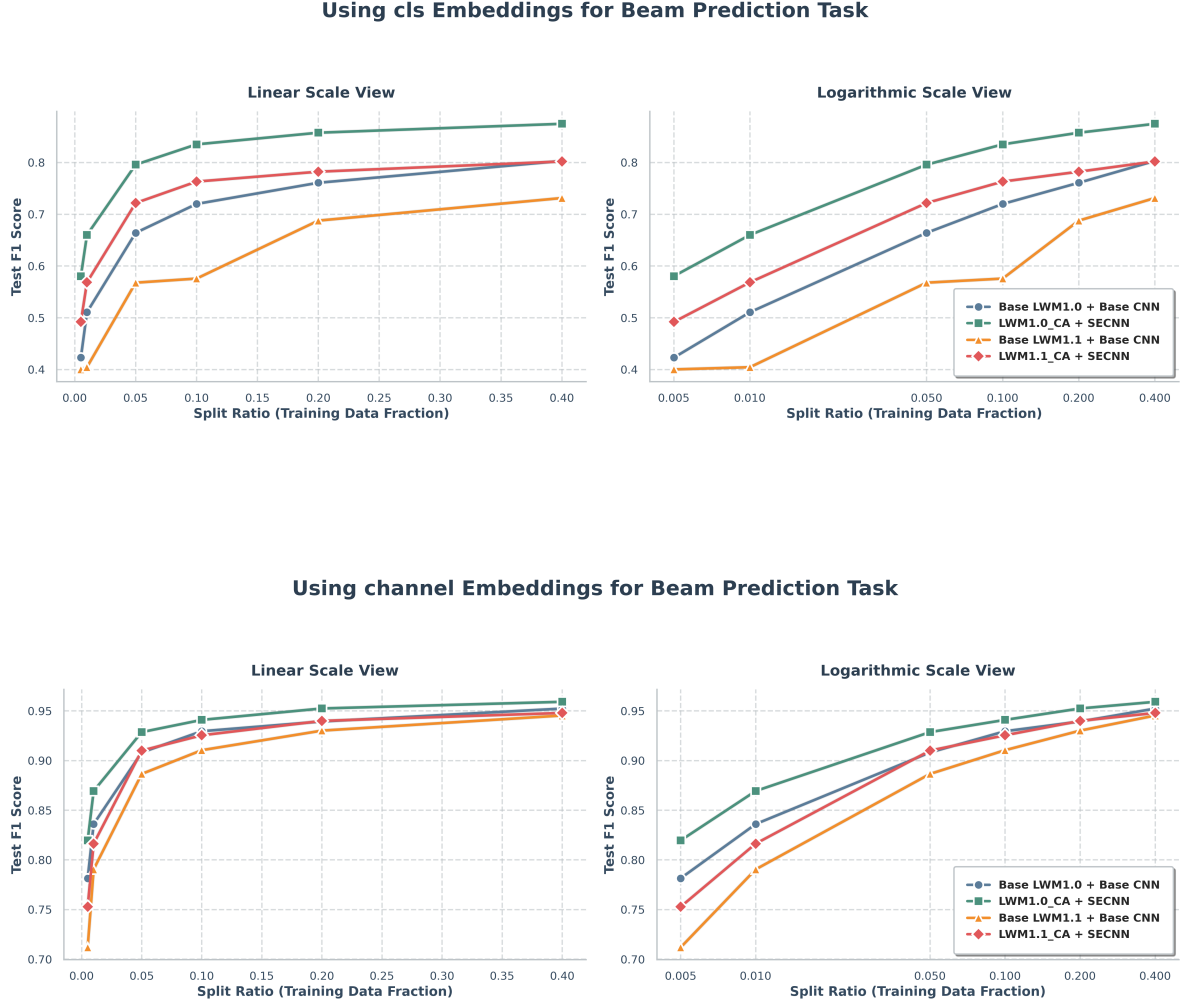


Figure 7: Generational performance comparison. Top: CLS embeddings. Bottom: Channel embeddings.

5.4 Codebase

- `lwm1_1/`: Vanilla LWM 1.1 backbone. `lwm_model.py` defines the 12-layer transformer ($d=128$, 8 heads). `1.1_script_Bp_CNN_base.py` runs downstream beam prediction using frozen embeddings.
- `lwm1_1_ca/`: CA-augmented variant. `torch_pipeline_v11.py` implements the end-to-end pre-training pipeline (channel $\rightarrow$ CA $\rightarrow$ patch $\rightarrow$ mask $\rightarrow$ LWM). `coordatt.py` defines the CA block. `benchmark_secnn.py` handles downstream evaluation. Pre-trained weights are stored as `512_25model1_1_weights_ca_e2e.pth` (best configuration).

6 Phase 4: Mask-Aware Transformer

Phases 1–3 improved the *downstream head* and *attention front-end* while keeping LWM’s standard 12-layer transformer backbone intact. Phase 4 takes the more ambitious step of **replacing the backbone entirely** with a Mask-Aware Transformer (MAT) [3], aiming to match LWM’s downstream accuracy at a fraction of the computational cost.

6.1 Architecture: MAT-Pseudo (8×16)

The successful variant, **MAT-Pseudo**, preserves LWM’s 128-patch tokenization while introducing mask-aware 2-D spatial attention via an 8×16 pseudo-grid reshape. The core building blocks are:

Window Multi-Head Attention (WindowMHA). Attention is computed within non-overlapping 4×4 spatial windows on the pseudo-grid. Invalid (masked) tokens receive a large negative bias ($-\tau$, $\tau=100$) before softmax, ensuring they are effectively ignored — this is the *mask-aware* property from [3]. Shifted windows (cyclic roll by $w/2$) are applied in alternating stages to enable cross-window information flow.

Adjusted Transformer Block (ATB). Each ATB stage performs: WindowMHA $\rightarrow$ channel concatenation with skip $\rightarrow 1 \times 1$ Conv fusion $\rightarrow$ MLP (expand $3 \times$, GELU) $\rightarrow 3 \times 3$ local convolution. Crucially, **no LayerNorm or BatchNorm** is used anywhere in the network. In the MAT design [3], normalization layers can leak spatial mask information across tokens, breaking the mask-aware inductive bias. This design choice is central to the architecture but also the root cause of the heavy variant’s challenges (Section 7).

Data Flow.

1. **Patching:** $(B, 2, 32, 32) \rightarrow (B, 128, 16)$ via LWM-compatible flattening.
2. **Masking:** 15% of patches masked (MCM-style symmetric real/imag).
3. **Embedding:** Linear projection $16 \rightarrow 64$ + learnable positional encoding.
4. **2-D Bridge:** $(B, 128, 64) \rightarrow (B, 64, 8, 16)$ pseudo-grid reshape.
5. **3-stage ATB:** mask-aware attention with dynamic mask relaxation after stage 1.
6. **Reverse Bridge:** $(B, 64, 8, 16) \rightarrow (B, 128, 64)$.
7. **Decoder:** gather masked embeddings, project $64 \rightarrow 16$, MSE loss.

For inference, Global Average Pooling on the pseudo-grid yields the CLS embedding $(B, 64)$, and flattening gives the channel embedding $(B, 128, 64)$ — both LWM-compatible.

6.2 Computational Efficiency

Table 7 presents the FLOP counts and parameter budgets measured via thop profiling (batch size = 1). MAT-Pseudo achieves a $2.3\times$ **reduction** in FLOPs relative to LWM 1.0 while using less than half the parameters.

Table 7: Computational cost comparison across all backbone variants.

Model	FLOPs (M)	Parameters (K)	Status
LWM 1.0	~ 75	~ 600	Baseline
LWM 1.1	~ 154	~ 2400	Baseline
MAT-Pseudo (8×16)	33.16	260	✓
MAT-Pseudo + CA	33.17	260	✓
MAT-Pseudo-Heavy	80.35	630	× (gradient explosion)
MAT-ViT (32×32)	264.45	259	× (lazy decoder)
MAT-ViT-Heavy	641.93	629	× (failed training)

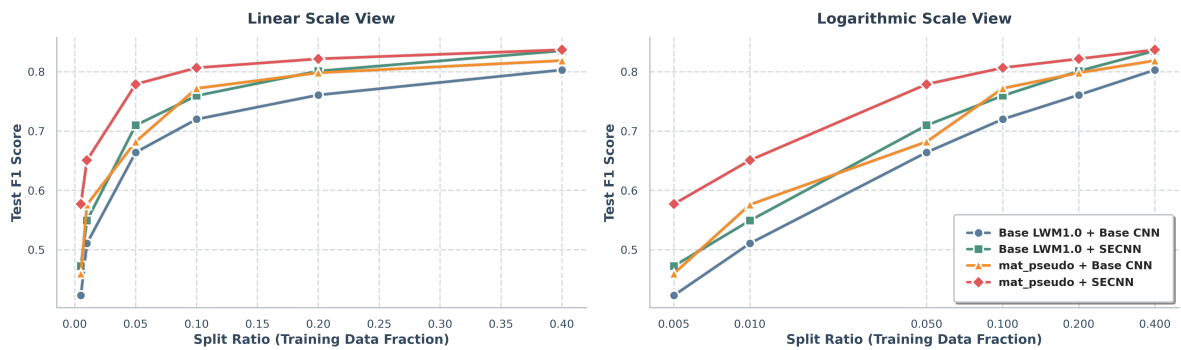
6.3 Results: MAT-Pseudo vs LWM

Table 8 compares MAT-Pseudo against the LWM 1.0 baselines. Both use a Base CNN downstream head.

Table 8: MAT-Pseudo vs LWM 1.0 beam prediction F1 scores.

Ratio (r)	CLS Embedding						Channel Embedding					
	0.5%	1%	5%	10%	20%	40%	0.5%	1%	5%	10%	20%	40%
Base LWM 1.0 + Base CNN	0.4231	0.5108	0.6642	0.7200	0.7609	0.8031	0.7815	0.8361	0.9082	0.9296	0.9394	0.9524
MAT-Pseudo + Base CNN	0.4598	0.5761	0.6821	0.7720	0.7983	0.8188	0.7327	0.8209	0.9089	0.9244	0.9380	0.9466

Using cls Embeddings for Beam Prediction Task



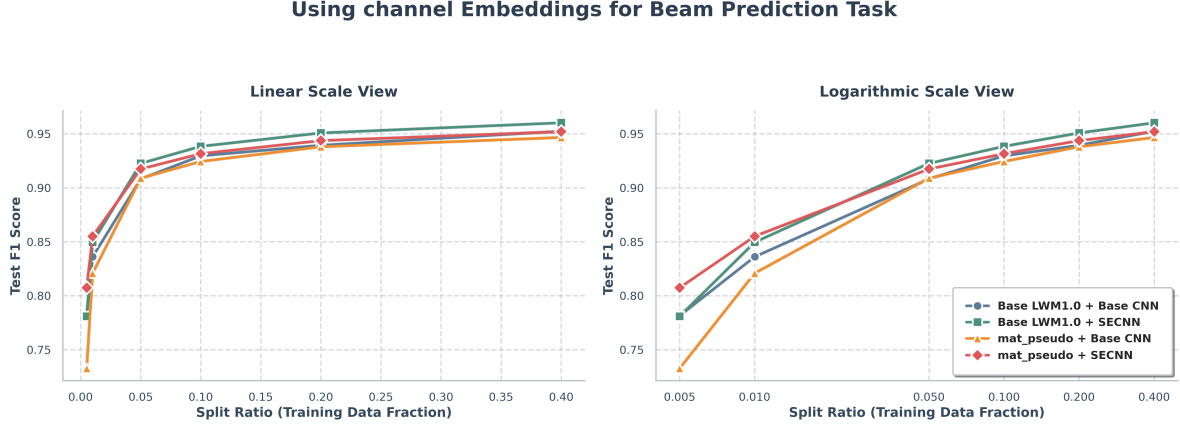


Figure 8: MAT-Pseudo vs LWM 1.0 (Base CNN head). Top: CLS. Bottom: Channel.

Inferences.

1. **CLS parity at $2.3\times$ less compute.** MAT-Pseudo matches or exceeds LWM 1.0 on CLS embeddings across all split ratios (+3.7 pp at $r=0.005$, +6.5 pp at $r=0.01$, +5.2 pp at $r=0.1$), despite using only 33M FLOPs vs LWM’s 75M.
2. **Channel embedding trade-off.** At the lowest split ratio ($r=0.005$), MAT-Pseudo trails LWM by -4.9 pp on channel embeddings. This gap closes rapidly: by $r=0.05$ the models are within 0.1 pp. The initial deficit is attributable to the smaller model capacity (260K vs 600K parameters) being insufficient to fully resolve the 128×64 channel manifold with minimal supervision.
3. **Efficiency frontier.** MAT-Pseudo establishes a new efficiency frontier: it is the only model in our evaluation that achieves LWM-competitive accuracy with sub-35M FLOPs, making it viable for edge deployment in latency-sensitive wireless systems.

6.4 CA Integration: MAT-Pseudo-CA

Following the established pattern from Phases 2 and 3, Coordinate Attention (Section 4.1, Figure 4) is integrated as a front-end to MAT-Pseudo. The CA module recalibrates the raw $(B, 2, 32, 32)$ channel before patching, adding only 76 parameters and negligible FLOPs (<0.01 M). Table 9 compares the CA-augmented variants.

Table 9: MAT-Pseudo-CA vs LWM-CA beam prediction F1 scores (SECNN head).

Ratio (r)	CLS Embedding						Channel Embedding					
	0.5%	1%	5%	10%	20%	40%	0.5%	1%	5%	10%	20%	40%
LWM 1.0_CA + SECNN	0.5805	0.6601	0.7959	0.8351	0.8576	0.8748	0.8196	0.8694	0.9287	0.9410	0.9525	0.9592
MAT-Pseudo-CA + SECNN	0.6219	0.6821	0.7917	0.8202	0.8390	0.8515	0.8139	0.8641	0.9213	0.9354	0.9452	0.9536

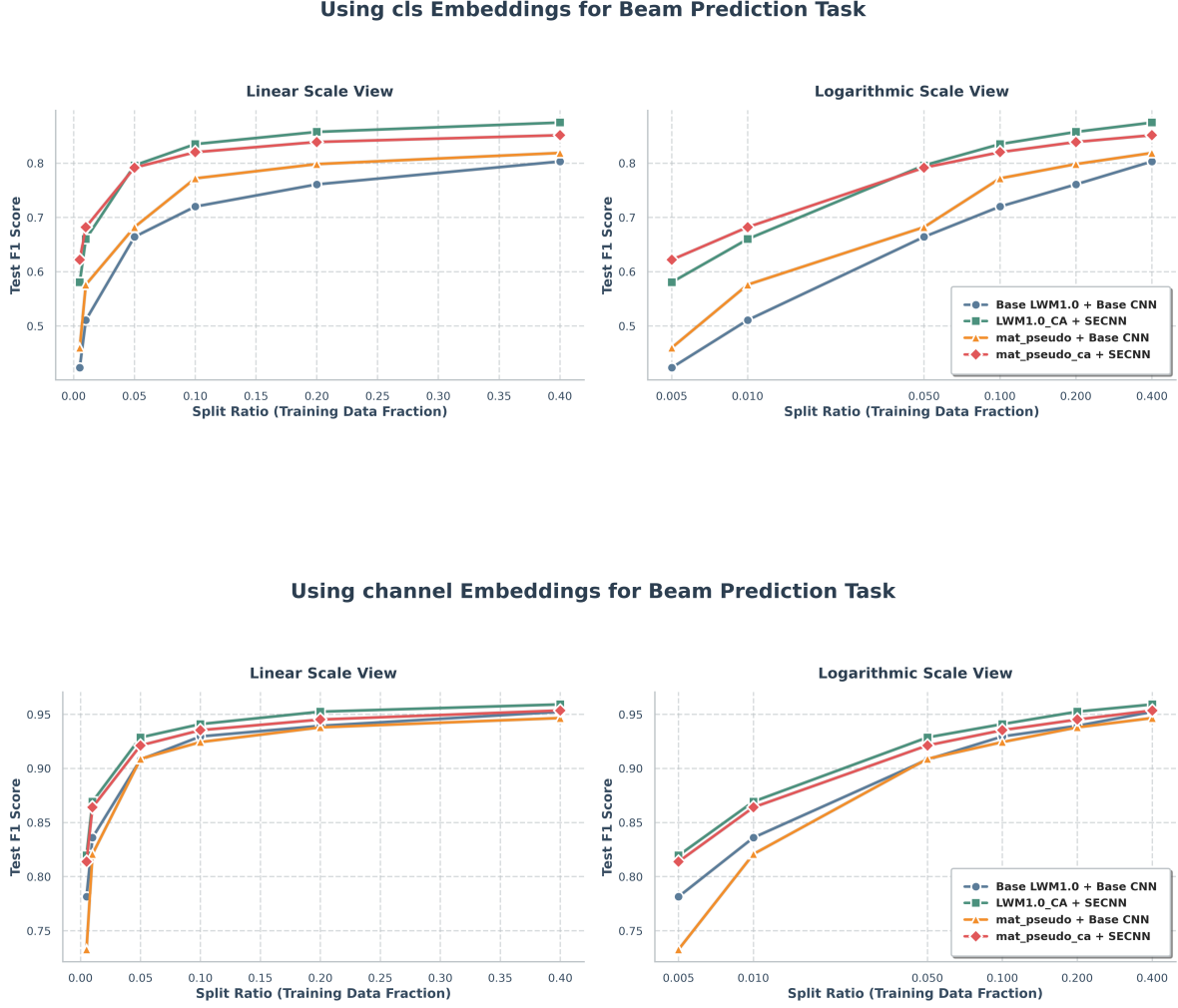


Figure 9: MAT-Pseudo-CA vs LWM-CA (SECNN head). Top: CLS. Bottom: Channel.

Inferences.

1. **Low-data superiority.** MAT-Pseudo-CA outperforms LWM-CA on CLS embeddings at the critical low-data points: +4.1 pp at $r=0.005$ and +2.2 pp at $r=0.01$. This is remarkable given that MAT-Pseudo operates at **less than half** the FLOPs of LWM 1.0.
2. **Convergence gap at high data.** At $r \geq 0.05$, LWM-CA retakes the lead by 0.4–2.3 pp. The larger capacity of LWM (600K vs 260K params) allows it to absorb more supervision. This suggests that a moderately scaled MAT variant (e.g., $d=96$) could close this gap while remaining well below LWM’s FLOP budget.
3. **Channel embedding near-parity.** On channel embeddings, both models are within ~ 0.6 pp across all splits, confirming that the 128×64 spatial embedding is rich enough to saturate the SECNN head regardless of the backbone’s total capacity.

Effect of CA on Pre-training Convergence. Figure 10 shows the pre-training loss curves for MAT-Pseudo with and without Coordinate Attention. The CA variant converges faster and to a lower final loss, demonstrating that the spatial priors injected

by CA improve not only downstream performance but also the quality of the self-supervised pre-training itself.

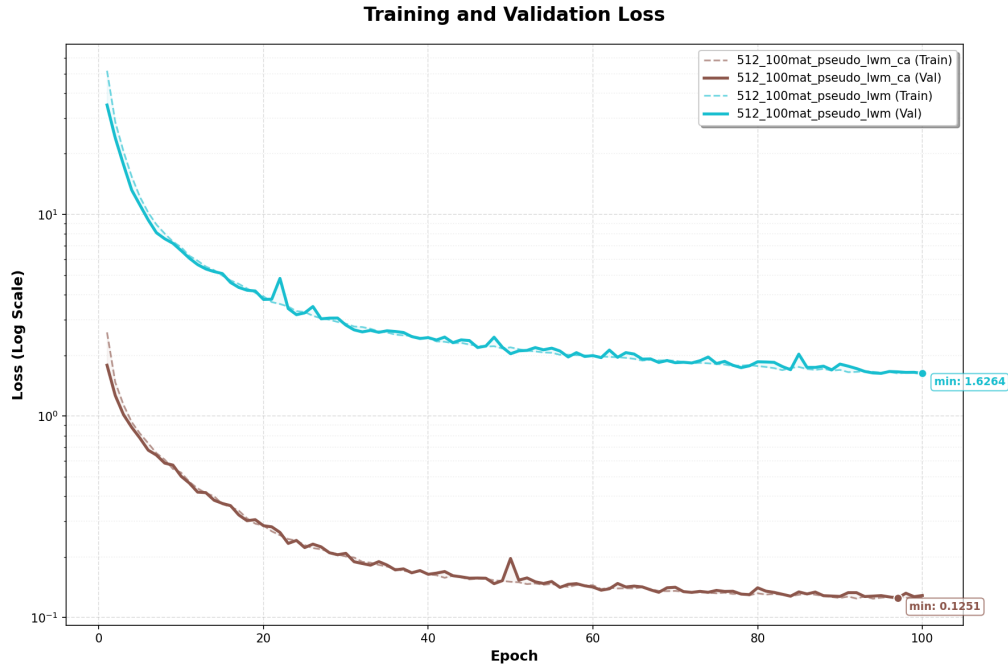


Figure 10: Pre-training loss comparison: MAT-Pseudo vs MAT-Pseudo-CA. The CA front-end accelerates convergence and reduces the final MCM loss.

6.5 Codebase

- `mask_aware_tf/`: Core MAT implementations. `mat_vit_lwm.py` defines WindowMHA and ATB. `mat_pseudo_lwm.py` implements the MAT-Pseudo model. `pretrain_pseudo.py` handles MCM pre-training. `mat_script_DownstreamBp.py` runs downstream evaluation. Weights: `512_100mat_pseudo_lwm_weights.pth`.
- `mat_pseudo_lwm_ca/`: CA-augmented MAT-Pseudo. `mat_pseudo_lwm_ca.py` extends MAT-Pseudo with a CoordAtt front-end. `pretrain_pseudo_ca.py` handles joint pre-training. `mat_ca_script_DownstreamBp.py` evaluates beam prediction. Weights: `512_100mat_pseudo_lwm_ca_weights.pth`.
- `mask_aware_tf_heavy/`: Scaled variants (MAT-Pseudo-Heavy, MAT-ViT-Heavy) used in the experimental exploration (Section 7).

All downstream scripts extract frozen embeddings from the pre-trained MAT backbone and train only the CNN head.

7 Architectural Exploration and Insights

During the development of the MAT backbone, four architectural variants were explored. Only MAT-Pseudo converged to useful representations. This section documents the remaining three variants and the design insights gained from each experiment.

7.1 MAT-Pseudo-Heavy

Goal: Scale MAT-Pseudo to match LWM’s depth (80M FLOPs, 630K parameters) for potentially higher accuracy.

Architecture: Identical to MAT-Pseudo but with increased d_{model} and additional ATB stages, roughly tripling the parameter count and FLOPs.

Failure mode: Catastrophic gradient explosion. The removal of all normalization layers (LayerNorm, BatchNorm) — necessary to prevent spatial mask leakage in the MAT architecture — left the deeper network without any mechanism to bound activation magnitudes. While the 3-stage MAT-Pseudo (260K params) can sustain stable gradients, the heavier variant rapidly diverges during pre-training, with loss values growing unbounded within the first few epochs. An attempt to mitigate this with DynamicTanh (a learnable per-channel tanh normalization) was also unsuccessful.

Downstream results confirm total failure: F1 scores plateau at ~ 0.35 – 0.43 across all split ratios (Table 7), indicating that the encoder learned no meaningful representations.

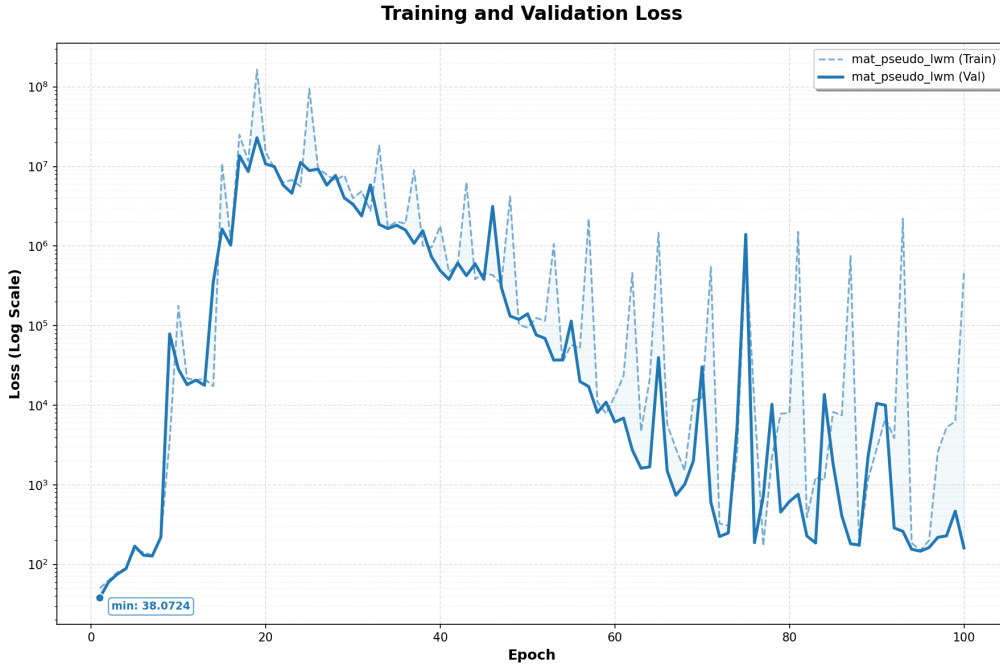


Figure 11: MAT-Pseudo-Heavy training loss. The loss diverges catastrophically due to the absence of normalization layers in the scaled architecture.

7.2 MAT-ViT (32×32)

Goal: Operate directly on the full 32×32 spatial grid (instead of the 8×16 pseudo-grid), potentially capturing finer spatial structure.

Architecture: Input $(B, 2, 32, 32) \rightarrow 1 \times 1$ Conv $\rightarrow (B, 64, 32, 32) \rightarrow$ spatial masking $\rightarrow$ 3-stage ATB $\rightarrow 1 \times 1$ Conv decoder $\rightarrow (B, 2, 32, 32)$. CLS embedding via GAP; channel embedding via AdaptiveAvgPool2d(8, 16).

Failure mode: Lazy decoder / encoder collapse. The pre-training loss converged to a low value (~ 0.3), suggesting the model was learning. However, downstream F1 scores were poor and erratic (e.g., channel embedding F1 dropping from 0.64 at $r=0.005$ to 0.42 at $r=0.01$, then recovering at higher splits). This non-monotonic behavior indicates that the *decoder* memorized reconstruction patterns while the *encoder* failed to extract robust, transferable features. A likely contributing factor is the absence of 2-D positional encodings: without explicit spatial priors, the 32×32 grid is too large for the 3-stage ATB to establish meaningful long-range dependencies.

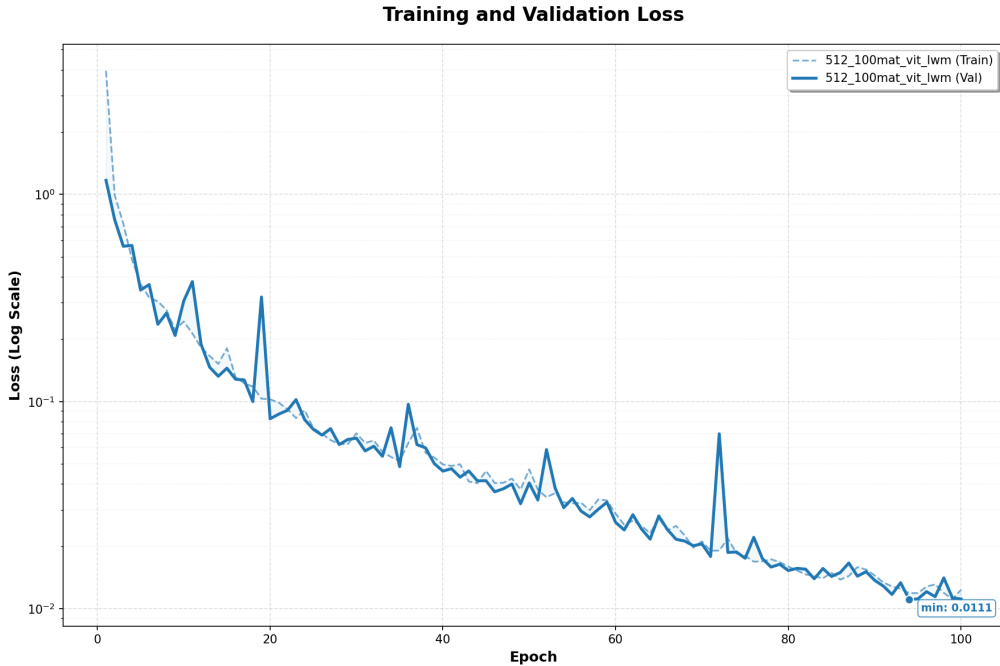


Figure 12: MAT-ViT pre-training loss. The loss converges smoothly, masking the underlying encoder collapse that is only revealed during downstream evaluation.

7.3 MAT-ViT-Heavy (32×32)

Goal: Scale MAT-ViT to 642M FLOPs / 629K parameters.

Failure: Combined both failure modes — the gradient instability from the normalization-free deep architecture and the lazy decoder problem from the 32×32 grid. Training was abandoned early.

7.4 Key Takeaways

1. **Normalization is critical for depth scaling.** The MAT constraint of no LN/BN fundamentally limits the viable model depth. Future work could explore alternatives such as RMSNorm (which operates per-token rather than spatially) or gradient clipping strategies.
2. **Pseudo-grid is the right abstraction.** The 8×16 pseudo-grid strikes the optimal balance between spatial resolution and the ATB's receptive field. The 32×32 grid is too large for 4×4 windows, leading to encoder collapse.
3. **Low pre-training loss $\neq$ good embeddings.** MAT-ViT's smooth convergence demonstrates that reconstruction loss alone is an insufficient proxy for downstream quality — the decoder can memorize without the encoder learning transferable features.

References

- [1] S. Alikhani, G. Charan, and A. Alkhateeb, “Large Wireless Model (LWM): A Foundation Model for Wireless Channels,” *arXiv preprint arXiv:2411.08872v1*, Nov. 2024.
- [2] S. Alikhani, G. Charan, and A. Alkhateeb, “Large Wireless Model (LWM): A Foundation Model for Wireless Channels,” *arXiv preprint arXiv:2411.08872v2*, Apr. 2025.
- [3] W. Li, Z. Lin, K. Zhou, L. Qi, Y. Wang, and J. Jia, “MAT: Mask-Aware Transformer for Large Hole Image Inpainting,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10758–10768, 2022.